

Image and Video Generations

Lecture 5: Zero-Shot Applications

劉育綸

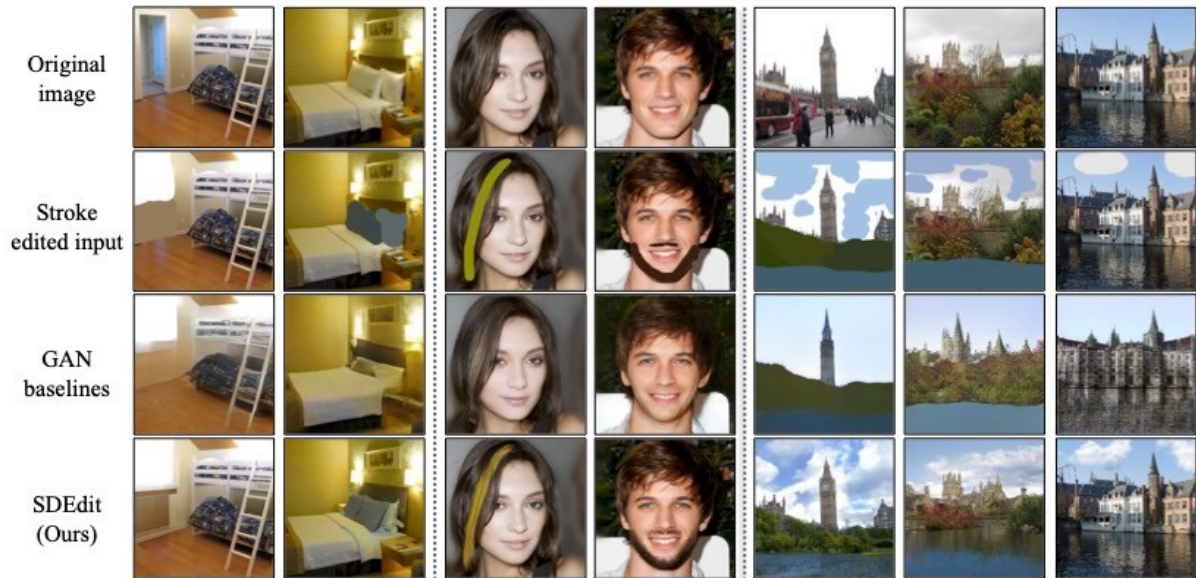
Yu-Lun (Alex) Liu



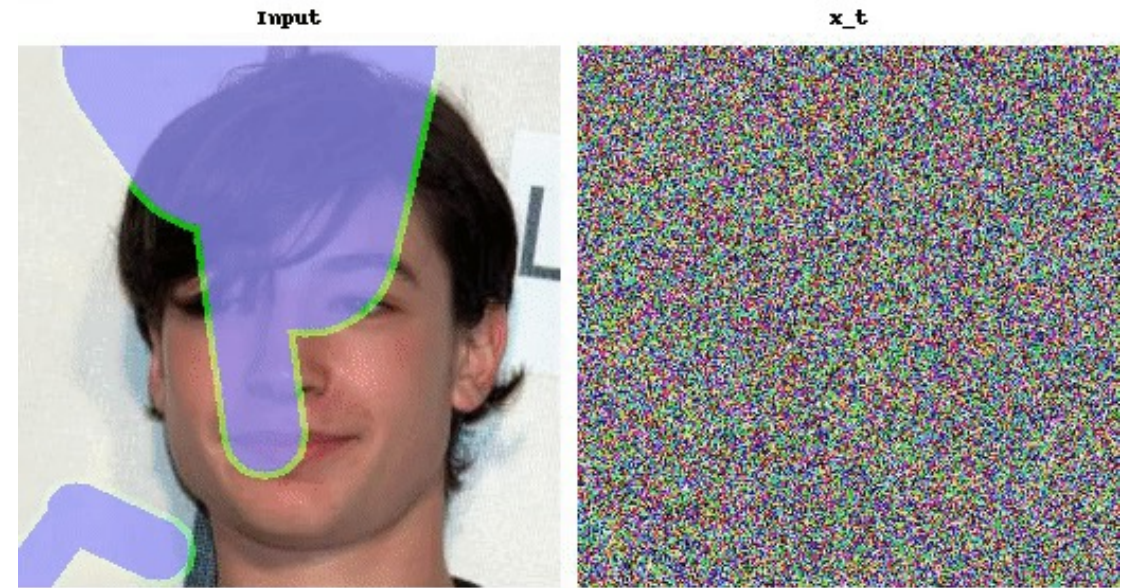
Zero-Shot Applications

Zero-Shot Adaptations

How to **edit** and **inpaint** images using a pretrained image diffusion model even without the need for fine-tuning?



Meng et al., SDEdit: Guided Image Synthesis and Editing with Stochastic Differential Equations, ICLR 2022.

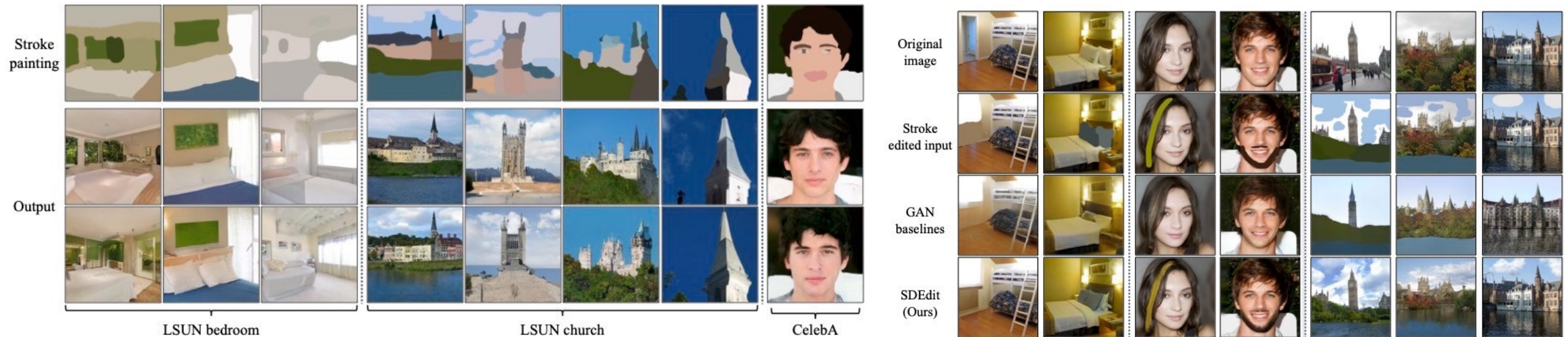


Lugmayr et al., RePaint: Inpainting using Denoising Diffusion Probabilistic Models, CVPR 2022.

SDEdit

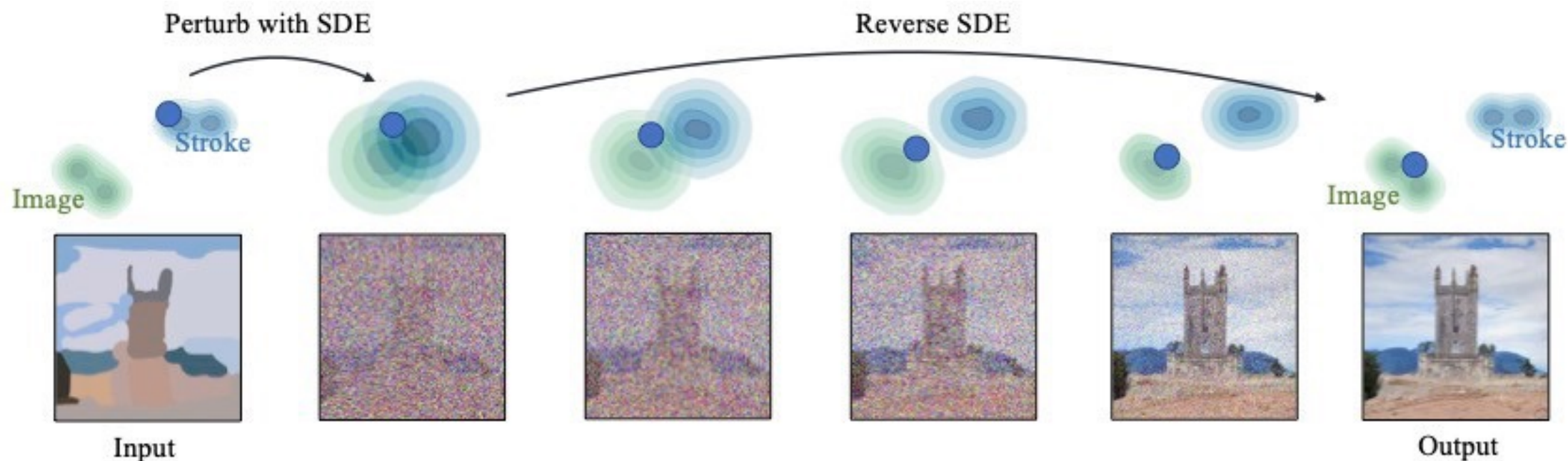
Image generation/editing through **user interaction**

- Image generation from sketches
- Image editing from scribbles



SDEdit

Perform the **forward** process for a bit and then **reverse** the process.

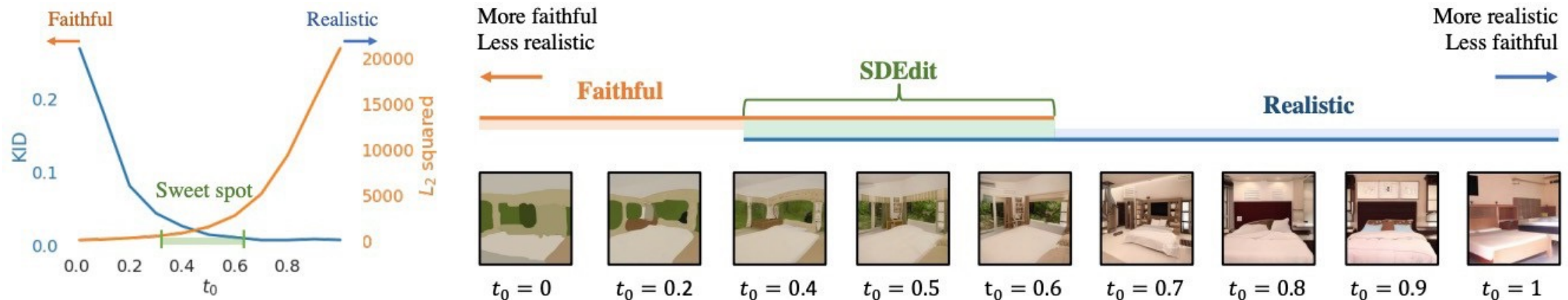


用 Noise 把 input 稍微淹沒，再讓 diffusion 去生成自然影像的細節

SDEdit

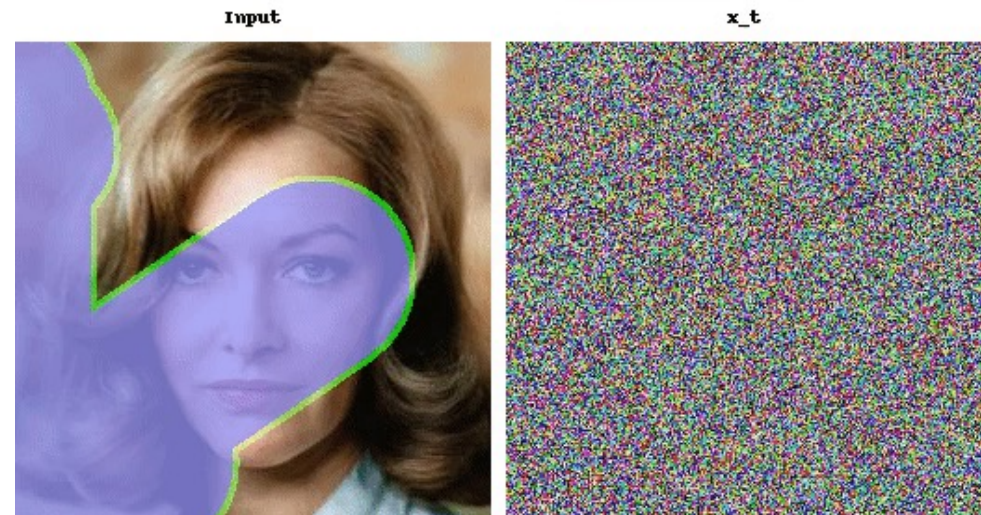
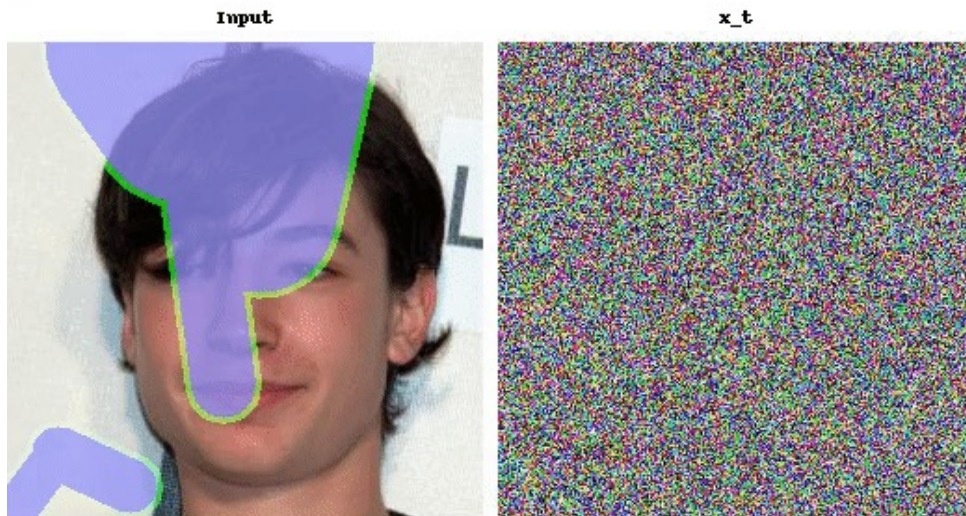
Realism vs. faithfulness

As you perform the forward process with a **longer** timestep, the output image becomes **more realistic but less faithful** (deviating from the given condition).



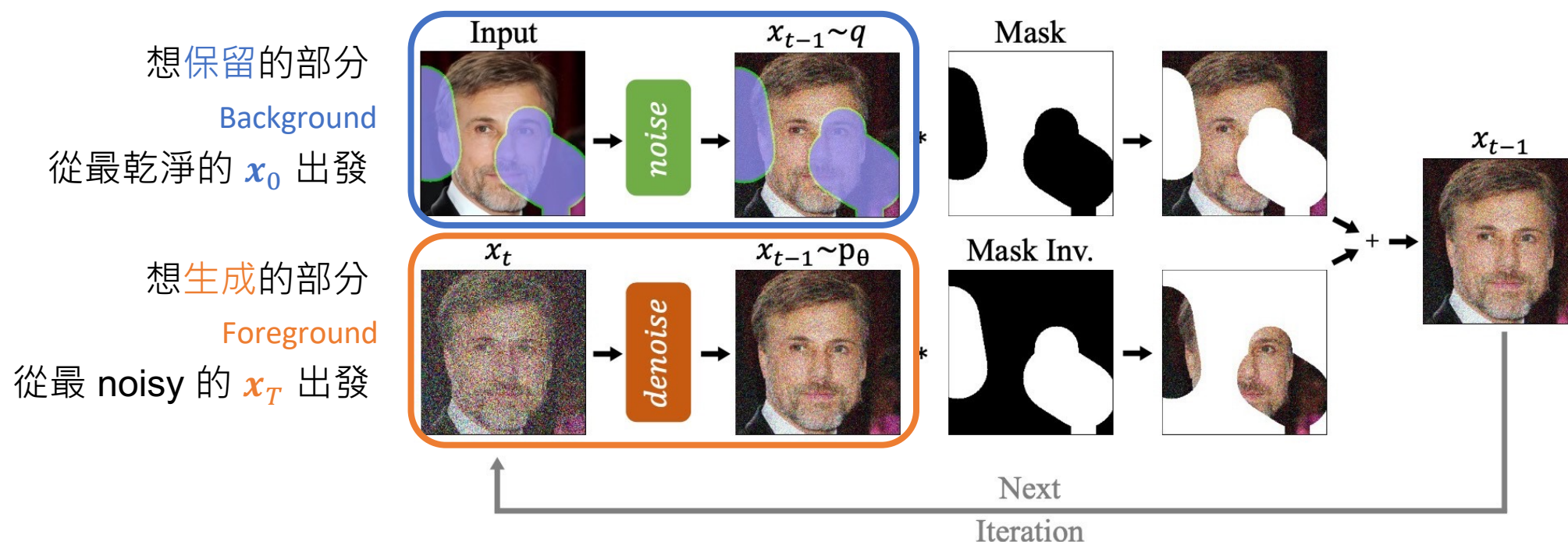
RePaint

- Filling regions in an image using a pretrained image diffusion model.
- Assume that the missing **foreground** is filled while the **background** is fixed.



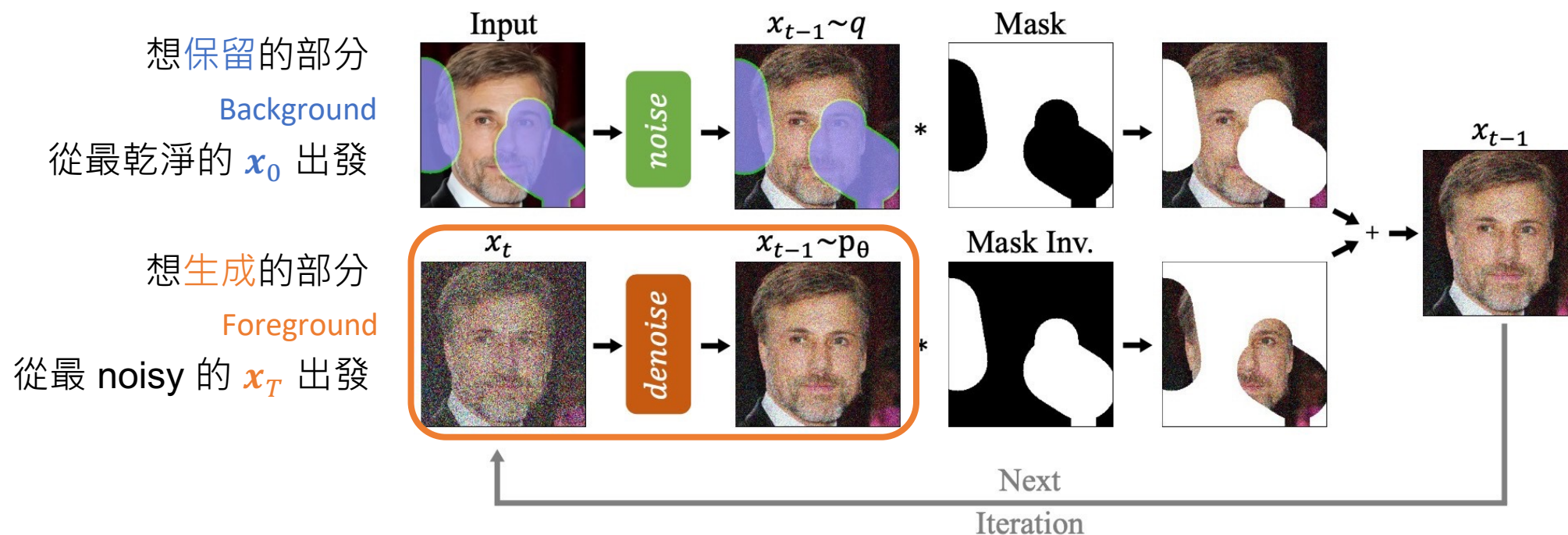
RePaint

Combine denoised **foreground** images (to be filled) and noisy **background** (to be fixed) images at each iteration of the reverse process.



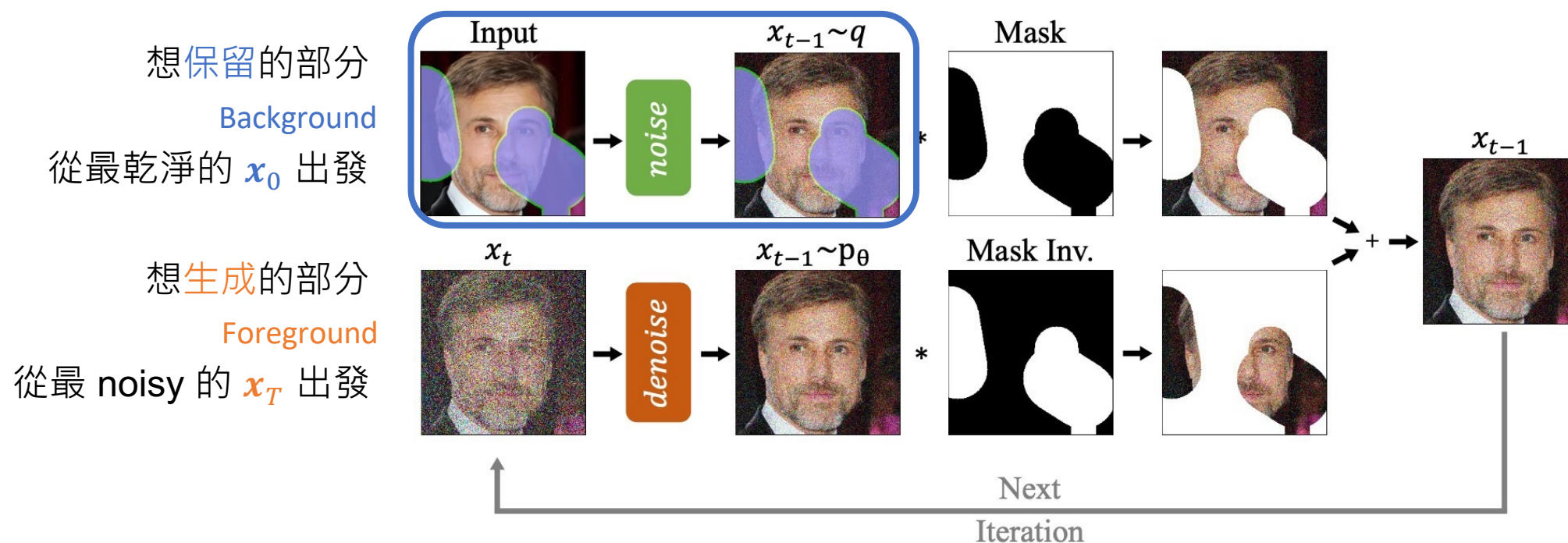
RePaint

1. Starting from x_T , at each timestep t , denoise x_t one step (reverse process), resulting x_{t-1}^f .



RePaint

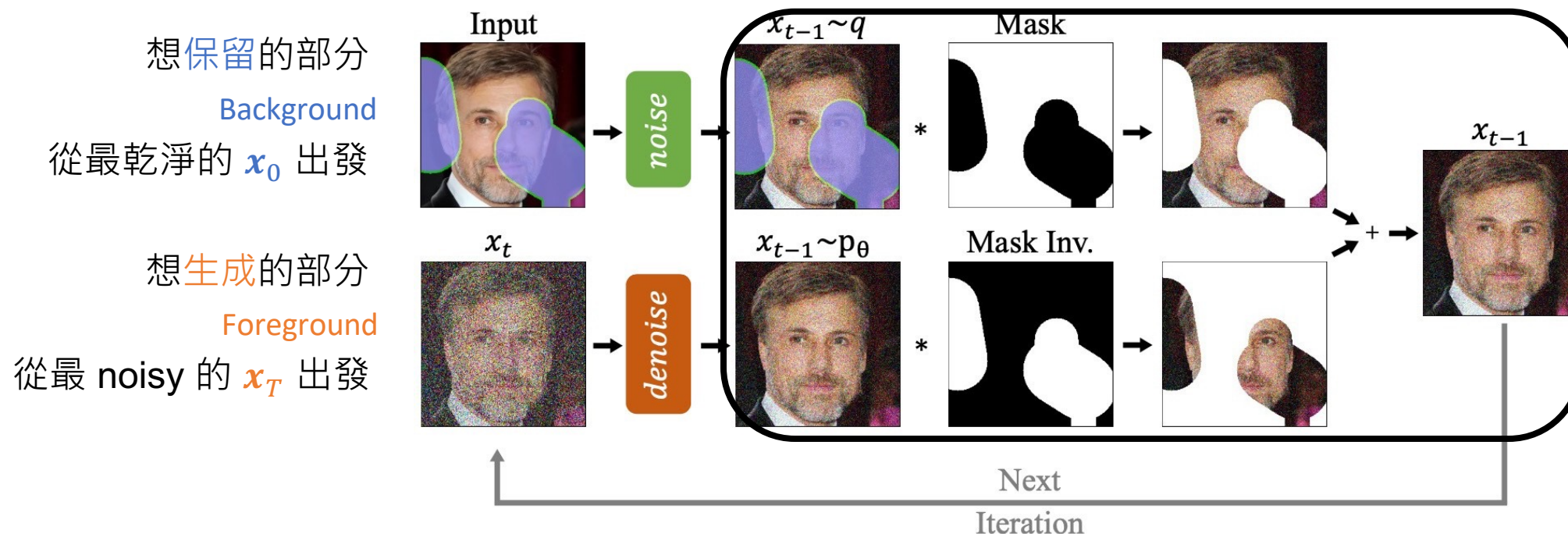
2. Perturb the input background image x_0^b (forward process) with a noise scale of the timestep $t - 1$, resulting x_{t-1}^b .



RePaint

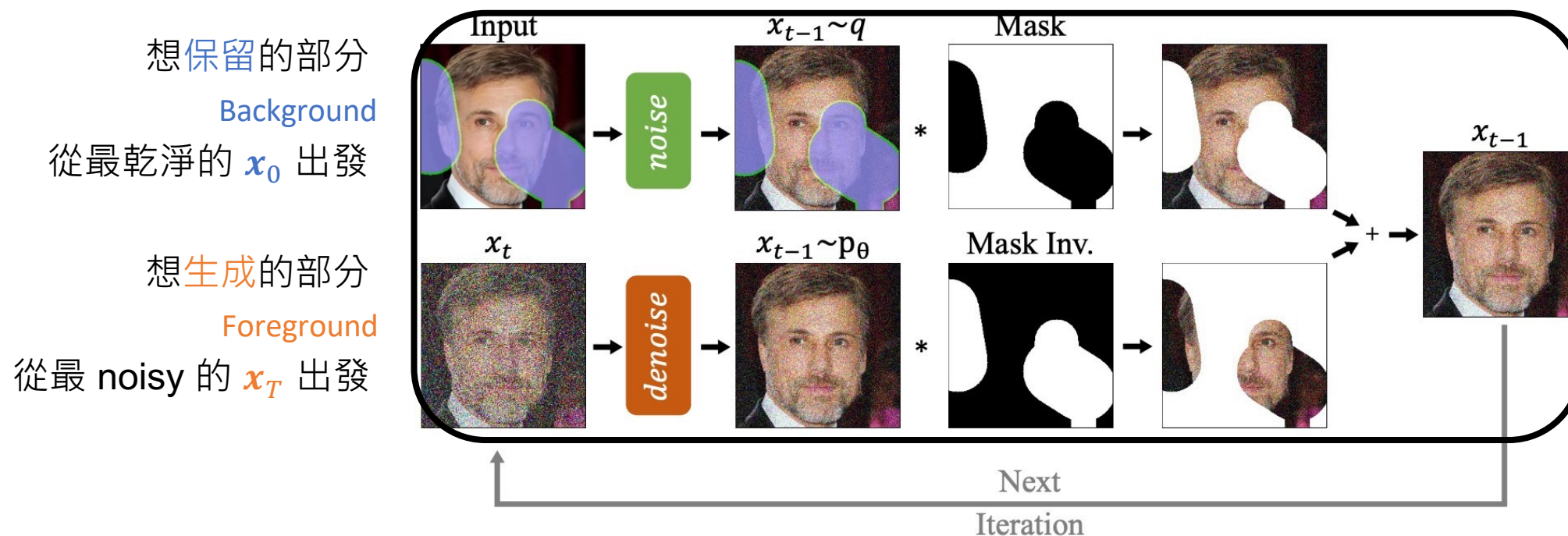
3. Combine x_{t-1}^b and x_{t-1}^f (where M is the background mask):

$$x_{t-1} = M \odot x_{t-1}^b + (1 - M) \odot x_{t-1}^f$$



RePaint

4. Repeat this process for $t = T, \dots, 1$.



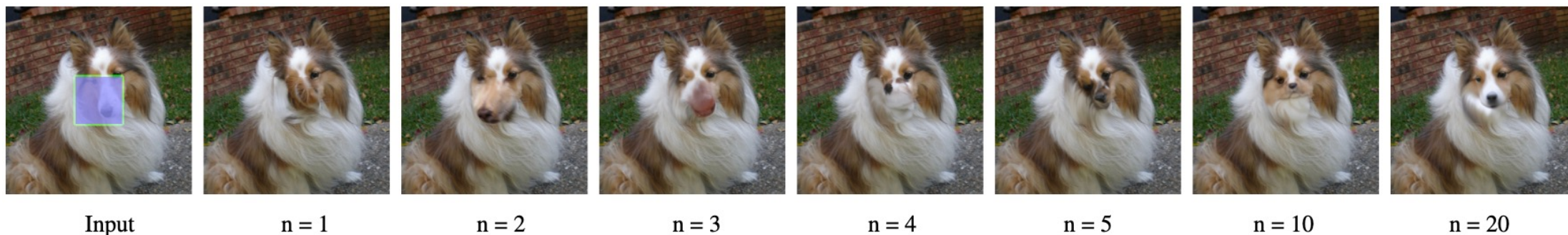
RePaint – Resampling

反覆 refine 邊界一致性的技巧，在每個 denoising time step t ，不是只 denoise 一次，而是重複 U 次“前進-後退”來改善 inpainting 效果

Algorithm 1 Inpainting using our RePaint approach.

1: $x_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$	
2: for $t = T, \dots, 1$ do	
3: for $u = 1, \dots, U$ do	U 次 resampling
4: $\epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ if $t > 1$, else $\epsilon = \mathbf{0}$	
5: $x_{t-1}^{\text{known}} = \sqrt{\bar{\alpha}_t}x_0 + (1 - \bar{\alpha}_t)\epsilon$	已知區域加 noise 到 $t - 1$
6: $z \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ if $t > 1$, else $z = \mathbf{0}$	
7: $x_{t-1}^{\text{unknown}} = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}} \epsilon_{\theta}(x_t, t) \right) + \sigma_t z$	未知區域 denoise 一步 ($t \rightarrow t - 1$)
8: $x_{t-1} = m \odot x_{t-1}^{\text{known}} + (1 - m) \odot x_{t-1}^{\text{unknown}}$	合併已知和未知區域
9: if $u < U$ and $t > 1$ then	
10: $x_t \sim \mathcal{N}(\sqrt{1 - \beta_{t-1}}x_{t-1}, \beta_{t-1}\mathbf{I})$	如果不是最後一個 iteration，重新加 noise 回 t
11: end if	
12: end for	
13: end for	
14: return x_0	

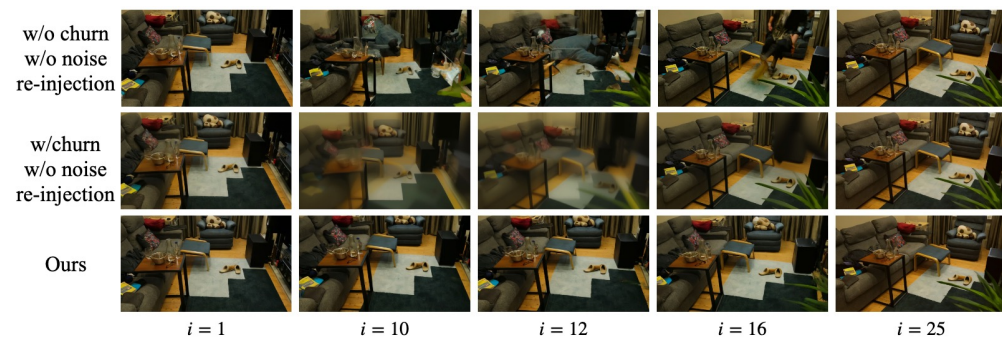
RePaint – Resampling



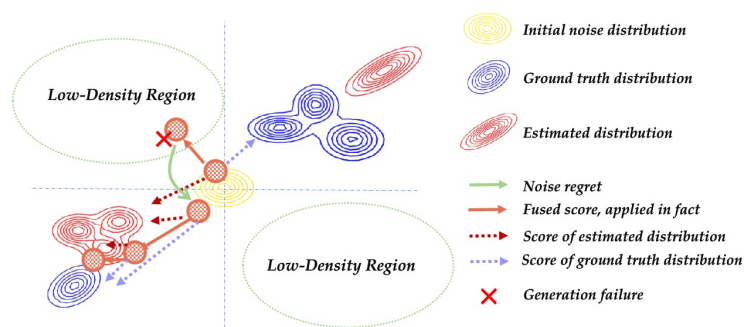
一般的 **DDIM** 反向去噪就 **50** 步，但步數這麼少不足以讓邊界協調
那就在同一個 **denoising time step** 反覆多做幾步

Resampling/Noise Re-Injection/Noise Regret

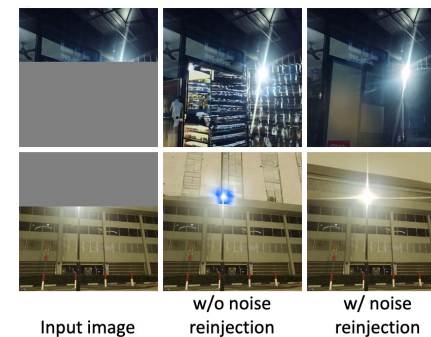
同樣的操作很常見，但有不同的名字



Time-Reversal [ECCV'24]
Noise re-injection



Be-Your-Outpainter [ECCV'24]
Noise regret

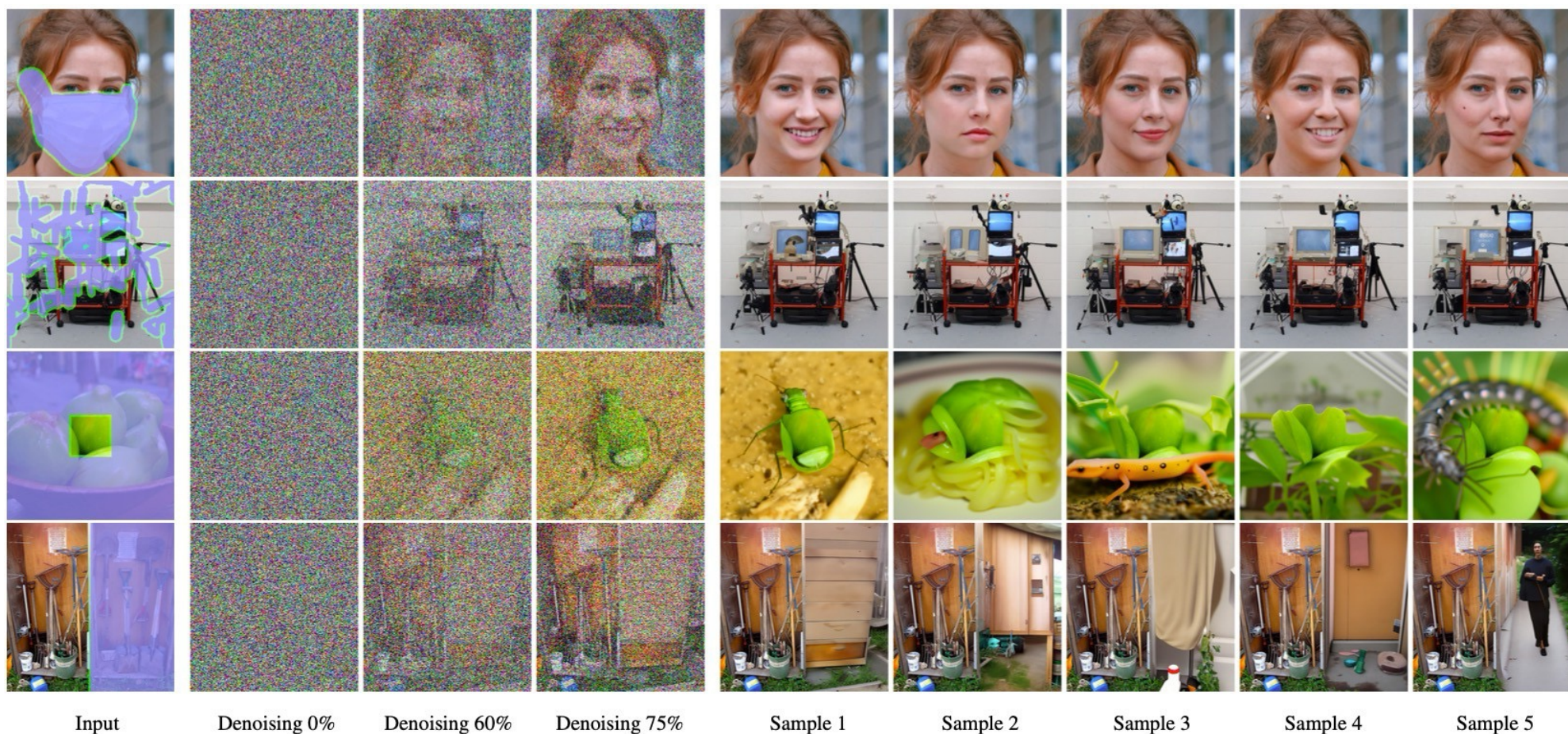


LightsOut [ICCV'25]
Noise reinjection

RePaint

不需任何訓練
就可以把一個原本目的是生成的 diffusion model
轉變任務為 inpainting

Results with different masks

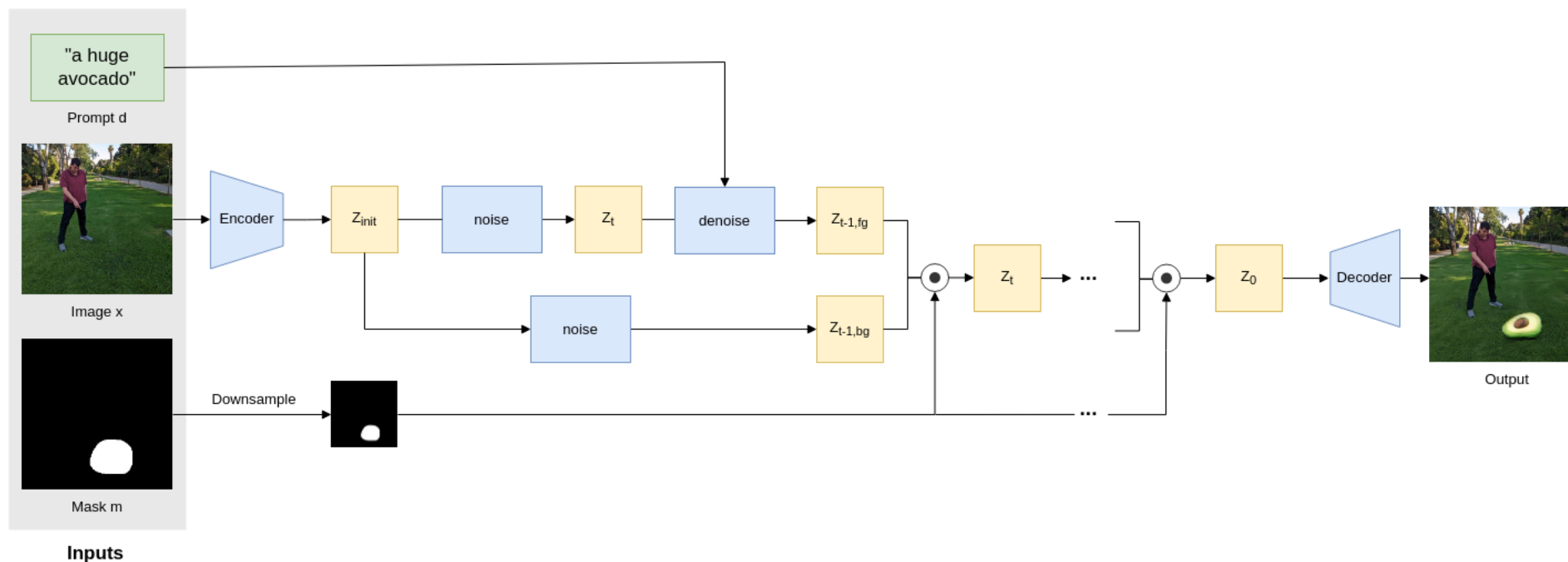


Outpainting

Blended Latent Diffusion

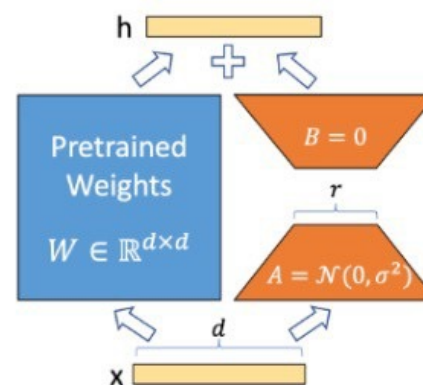
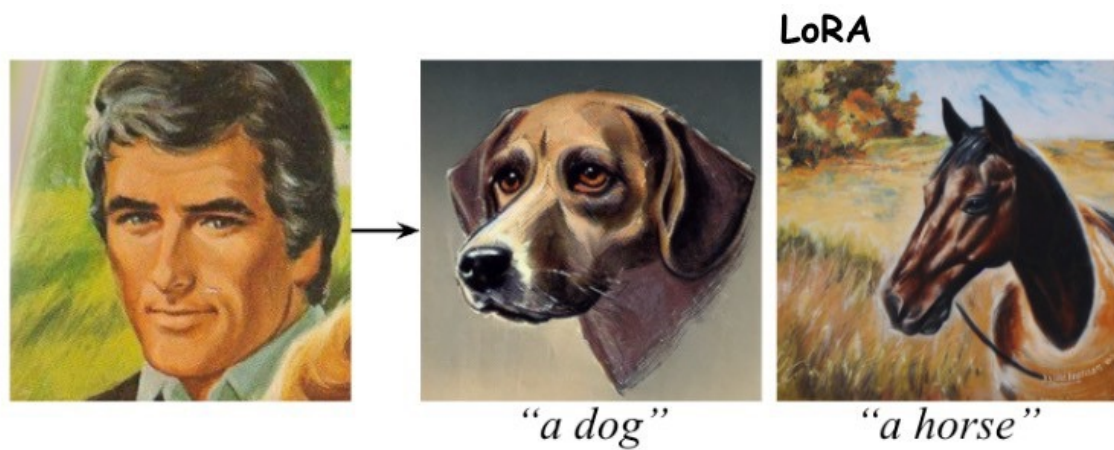


Blended Latent Diffusion



基本上就是 latent 版本的 RePaint

Assignment 2



Diffusers



license Apache-2.0

release v0.30.3

downloads/month 3M

Contributor Covenant 2.1

Follow @diffuserslib

🤗 Diffusers is the go-to library for state-of-the-art pretrained diffusion models for generating images, audio, and even 3D structures of molecules. Whether you're looking for a simple inference solution or training your own diffusion models, 🤗 Diffusers is a modular toolbox that supports both. Our library is designed with a focus on [usability over performance](#), [simple over easy](#), and [customizability over abstractions](#).

🤗 Diffusers offers three core components:

- State-of-the-art [diffusion pipelines](#) that can be run in inference with just a few lines of code.
- Interchangeable noise [schedulers](#) for different diffusion speeds and output quality.
- Pretrained [models](#) that can be used as building blocks, and combined with schedulers, for creating your own end-to-end diffusion systems.